System Design Masterclass Vol. 2: Low-Level Design (LLD) — Machine Coding and Concurrency in Java

Abstract

A production-grade, interview-ready engineering masterclass on Low-Level Design (LLD) Machine Coding and Java Concurrency, designed for Days 8–15 of the 30-day intensive curriculum. This volume tackles the hardest part of machine coding interviews: writing clean, modular, extensible, and thread-safe object-oriented systems under timed 90-minute conditions. It covers Java concurrency primitives (ReentrantLock, tryLock, AtomicInteger, volatile, ExecutorService, and the Producer-Consumer pattern with BlockingQueue) followed by four complete, compilable, end-to-end case studies: Multi-Floor Parking Lot System, BookMyShow Concurrent Seat Reservation with auto-release timeout timers, Splitwise Expense Sharing Engine with Equal/Exact/Percent strategies, and a thread-safe O(1) LRU Cache implemented via custom Doubly Linked List and HashMap.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis

1 Introduction: Machine Coding & Concurrency in Java

Week 2 (Days 8 to 15) represents the bridge between object-oriented design patterns and working, production-grade software. In 90-minute machine coding rounds conducted by top tech firms (Flipkart, Uber, Swiggy, Amazon, PhonePe), interviewers evaluate your ability to write clean, modular, extensible, and thread-safe Java code under tight time constraints.

This volume covers two essential pillars:

1) Core Java Concurrency & Multithreading: Thread pools, synchronized methods, ReentrantLock, AtomicInteger, volatile variables, and the Producer-Consumer pattern.
2) Four Complete Machine Coding Implementations: Fully designed, end-to-end runnable systems for Parking Lot, BookMyShow (Concurrent Seat Reservation), Splitwise (Expense Sharing), and LRU Cache.

2 Java Concurrency Essentials for Machine Coding

Machine coding evaluations frequently introduce concurrent user actions (e.g., two users booking the same movie seat, or two cars claiming the same parking spot). You must know how to prevent race conditions without causing deadlocks.

2.1 1. Synchronized vs. ReentrantLock

- synchronized: Intrinsic lock tied to an object monitor. Automatically released when exiting the block or method. Simple, but lacks timeouts and fairness control.
- ReentrantLock: Explicit lock from java.util.concurrent.locks. Allows tryLock(timeout, timeUnit), fair locking policies, and multiple condition variables.

```
import java.util.concurrent.locks.ReentrantLock;
import java.util.concurrent.TimeUnit;

public class SeatLockManager {
    private final ReentrantLock lock = new ReentrantLock();

    public boolean bookWithTimeout(String seatId, long timeoutMillis) {
        try {
            // Attempt to acquire lock without blocking indefinitely
            if (lock.tryLock(timeoutMillis, TimeUnit.MILLISECONDS)) {
                try {
                    // Critical section: reserve seat
                    System.out.println("Seat " + seatId + " successfully locked.");
                    return true;
                } finally {
```

```
                lock.unlock(); // Always release in finally block!
            }
        } else {
            System.out.println("Could not acquire lock for seat " + seatId + " (Timeout)");
            return false;
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        return false;
    }
  }
}
```

2.2 2. Atomic Variables & Volatile

- volatile: Guarantees memory visibility across threads (reads and writes go directly to main memory, bypassing CPU L1/L2 caches). Does NOT guarantee atomicity for compound operations (e.g., count++).
- AtomicInteger / AtomicLong: Employs hardware-level Compare-And-Swap (CAS) instructions to perform lock-free, atomic increments and updates.

```
import java.util.concurrent.atomic.AtomicInteger;

public class ParkingSpotCounter {
  private final AtomicInteger availableSpots;

  public ParkingSpotCounter(int total) {
    this.availableSpots = new AtomicInteger(total);
  }

  public boolean tryOccupySpot() {
    while (true) {
      int current = availableSpots.get();
      if (current <= 0) return false;
      // Lock-free atomic decrement
      if (availableSpots.compareAndSet(current, current - 1)) {
        return true;
      }
    }
  }

  public void releaseSpot() {
    availableSpots.incrementAndGet();
  }
}
```

2.3 3. The Classic Producer-Consumer Pattern

In event-driven machine coding (e.g., processing orders or notifications), the Producer-Consumer pattern decouples event generation from background workers using BlockingQueue.

```
import java.util.concurrent.*;

public class OrderProcessingPipeline {
  private final BlockingQueue<String> orderQueue = new ArrayBlockingQueue<>(100);

  public void startWorkers(int workerCount) {
    ExecutorService executor = Executors.newFixedThreadPool(workerCount);
    for (int i = 0; i < workerCount; i++) {
      executor.submit(() -> {
        while (!Thread.currentThread().isInterrupted()) {
          try {
            String order = orderQueue.take(); // Blocks if empty
            System.out.println(Thread.currentThread().getName() + " processed: " + order);
```

```
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    break;
                }
            }
        });
    }
}

public void submitOrder(String orderId) throws InterruptedException {
    orderQueue.put(orderId); // Blocks if full
}
}
```

3 Case Study 1: Multi-Floor Parking Lot System

3.1 Requirements & Scope

- Support multiple vehicle types: BIKE, CAR, TRUCK.
- Multiple parking floors, each with dedicated spots for each vehicle type.
- Parking strategy: Default to lowest floor, nearest available spot.
- Issues a ticket on entry with timestamp, spot number, and vehicle details.
- Calculates fee on exit based on duration and vehicle type.

3.2 Clean Java Implementation

```
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

// 1. Domain Models
enum VehicleType { BIKE, CAR, TRUCK }

class Vehicle {
    private final String licensePlate;
    private final VehicleType type;
    public Vehicle(String plate, VehicleType type) {
        this.licensePlate = plate;
        this.type = type;
    }
    public VehicleType getType() { return type; }
    public String getLicensePlate() { return licensePlate; }
}

class ParkingSpot {
    private final int spotId;
    private final VehicleType supportedType;
    private boolean isFree = true;
    private Vehicle parkedVehicle;

    public ParkingSpot(int id, VehicleType type) {
        this.spotId = id;
        this.supportedType = type;
    }

    public synchronized boolean assignVehicle(Vehicle v) {
        if (!isFree || v.getType() != supportedType) return false;
        this.parkedVehicle = v;
        this.isFree = false;
        return true;
    }

    public synchronized void vacate() {
```

```
        this.parkedVehicle = null;
        this.isFree = true;
    }

    public boolean isFree() { return isFree; }
    public int getSpotId() { return spotId; }
    public VehicleType getSupportedType() { return supportedType; }
}

class Ticket {
    private final String ticketId;
    private final Vehicle vehicle;
    private final ParkingSpot spot;
    private final long entryTime;

    public Ticket(Vehicle vehicle, ParkingSpot spot) {
        this.ticketId = UUID.randomUUID().toString().substring(0, 8);
        this.vehicle = vehicle;
        this.spot = spot;
        this.entryTime = System.currentTimeMillis();
    }

    public String getTicketId() { return ticketId; }
    public ParkingSpot getSpot() { return spot; }
    public Vehicle getVehicle() { return vehicle; }
    public long getEntryTime() { return entryTime; }
}

// 2. Fee Calculation Strategy (OCP/Strategy Pattern)
interface PricingStrategy {
    double calculateFee(long durationMillis, VehicleType type);
}

class FlatRatePricingStrategy implements PricingStrategy {
    public double calculateFee(long durationMillis, VehicleType type) {
        long hours = Math.max(1, durationMillis / (1000 * 60 * 60));
        switch (type) {
            case BIKE: return hours * 10.0;
            case CAR: return hours * 20.0;
            case TRUCK: return hours * 50.0;
            default: return hours * 15.0;
        }
    }
}

// 3. Central Service
class ParkingLotManager {
    private final List<ParkingSpot> spots = new ArrayList<>();
    private final Map<String, Ticket> activeTickets = new ConcurrentHashMap<>();
    private final PricingStrategy pricingStrategy;

    public ParkingLotManager(PricingStrategy pricing) {
        this.pricingStrategy = pricing;
    }

    public void addSpot(ParkingSpot spot) { spots.add(spot); }

    public synchronized Ticket parkVehicle(Vehicle v) {
        for (ParkingSpot spot : spots) {
            if (spot.isFree() && spot.getSupportedType() == v.getType()) {
                if (spot.assignVehicle(v)) {
```

```
                Ticket ticket = new Ticket(v, spot);
                activeTickets.put(ticket.getTicketId(), ticket);
                System.out.println("Parked " + v.getLicensePlate() + " at Spot " + spot.getSpotId());
                return ticket;
            }
        }
    }
    System.out.println("Parking Full for type: " + v.getType());
    return null;
}

public synchronized double unparkVehicle(String ticketId) {
    Ticket ticket = activeTickets.remove(ticketId);
    if (ticket == null) throw new IllegalArgumentException("Invalid Ticket ID");

    ticket.getSpot().vacate();
    long duration = System.currentTimeMillis() - ticket.getEntryTime() + 3600000; // Simulated 1 hr
    double fee = pricingStrategy.calculateFee(duration, ticket.getVehicle().getType());
    System.out.println("Unparked " + ticket.getVehicle().getLicensePlate() + " | Fee: Rs." + fee);
    return fee;
}
}
```

4 Case Study 2: BookMyShow (Concurrent Seat Reservation)

4.1 The Concurrency Challenge

The primary challenge in BookMyShow is preventing double-booking: when two users attempt to book the exact same seat simultaneously.

Seat Lifecycle State Machine:

$$\text{AVAILABLE} \xrightarrow{\text{User clicks seat}} \text{TEMPORARILY HELD (10 mins)} \xrightarrow{\text{Payment Success}} \text{BOOKED}$$

If payment fails or the 10-minute timer expires, the seat automatically transitions back to AVAILABLE.

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.locks.ReentrantLock;

enum SeatStatus { AVAILABLE, HELD, BOOKED }

class Seat {
    private final String seatId;
    private SeatStatus status = SeatStatus.AVAILABLE;
    private String lockedByUserId = null;
    private final ReentrantLock lock = new ReentrantLock();

    public Seat(String id) { this.seatId = id; }

    public boolean lockSeat(String userId) {
        lock.lock();
        try {
            if (status == SeatStatus.AVAILABLE) {
                this.status = SeatStatus.HELD;
                this.lockedByUserId = userId;
                return true;
            }
            return false;
        } finally {
            lock.unlock();
        }
    }
```

```
    public boolean confirmBooking(String userId) {
        lock.lock();
        try {
            if (status == SeatStatus.HELD && userId.equals(lockedByUserId)) {
                this.status = SeatStatus.BOOKED;
                return true;
            }
            return false;
        } finally {
            lock.unlock();
        }
    }

    public void releaseSeat(String userId) {
        lock.lock();
        try {
            if (status == SeatStatus.HELD && userId.equals(lockedByUserId)) {
                this.status = SeatStatus.AVAILABLE;
                this.lockedByUserId = null;
            }
        } finally {
            lock.unlock();
        }
    }

    public SeatStatus getStatus() { return status; }
    public String getSeatId() { return seatId; }
}

class BookingService {
    private final Map<String, Seat> seats = new ConcurrentHashMap<>();
    private final ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(2);

    public void addSeat(String seatId) { seats.put(seatId, new Seat(seatId)); }

    public boolean reserveSeat(String seatId, String userId, long holdTimeoutSeconds) {
        Seat seat = seats.get(seatId);
        if (seat == null) return false;

        boolean locked = seat.lockSeat(userId);
        if (locked) {
            System.out.println("User " + userId + " HELD seat " + seatId + " for " + holdTimeoutSeconds + "s");
            // Schedule auto-release if payment is not completed in time
            scheduler.schedule(() -> {
                if (seat.getStatus() == SeatStatus.HELD) {
                    seat.releaseSeat(userId);
                    System.out.println("Auto-released seat " + seatId + " due to timeout.");
                }
            }, holdTimeoutSeconds, TimeUnit.SECONDS);
            return true;
        }
        return false;
    }

    public boolean completePayment(String seatId, String userId) {
        Seat seat = seats.get(seatId);
        if (seat != null && seat.confirmBooking(userId)) {
            System.out.println("Booking CONFIRMED for user " + userId + " on seat " + seatId);
            return true;
        }
        System.out.println("Booking FAILED for seat " + seatId);
```

```
        return false;
    }
}
```

5 Case Study 3: Splitwise (Expense Sharing Engine)

5.1 Requirements & Split Types

- Users can create groups and add shared expenses.
- Three split modes:
    1) EQUAL: Cost divided uniformly among participants.
    2) EXACT: Each participant owes an explicit currency value (sum must equal total).
    3) PERCENT: Each participant owes a percentage (sum must equal 100%).
- Maintains a balance sheet showing who owes whom.

```
import java.util.*;

enum SplitType { EQUAL, EXACT, PERCENT }

class Split {
    private final String userId;
    protected double amount;

    public Split(String userId) { this.userId = userId; }
    public String getUserId() { return userId; }
    public double getAmount() { return amount; }
    public void setAmount(double amount) { this.amount = amount; }
}

class EqualSplit extends Split { public EqualSplit(String id) { super(id); } }
class ExactSplit extends Split {
    public ExactSplit(String id, double amount) { super(id); this.amount = amount; }
}

interface ExpenseValidator {
    boolean validate(double totalAmount, List<Split> splits);
}

class EqualExpenseValidator implements ExpenseValidator {
    public boolean validate(double totalAmount, List<Split> splits) {
        double splitAmount = Math.round((totalAmount / splits.size()) * 100.0) / 100.0;
        for (Split s : splits) s.setAmount(splitAmount);
        return true;
    }
}

class SplitwiseManager {
    // balanceSheet: UserA -> (UserB -> amount A owes B)
    private final Map<String, Map<String, Double>> balanceSheet = new HashMap<>();

    public void addExpense(String paidBy, double totalAmount, List<Split> splits, ExpenseValidator validator) {
        if (!validator.validate(totalAmount, splits)) {
            throw new IllegalArgumentException("Invalid split configuration!");
        }

        for (Split split : splits) {
            String owedUser = split.getUserId();
            if (owedUser.equals(paidBy)) continue;

            double amount = split.getAmount();
            // Update balanceSheet: owedUser owes paidBy
```

```
            balanceSheet.putIfAbsent(owedUser, new HashMap<>());
            Map<String, Double> balances = balanceSheet.get(owedUser);
            balances.put(paidBy, balances.getOrDefault(paidBy, 0.0) + amount);
        }
    }

    public void showBalances() {
        boolean hasBalances = false;
        for (String userA : balanceSheet.keySet()) {
            for (Map.Entry<String, Double> entry : balanceSheet.get(userA).entrySet()) {
                if (entry.getValue() > 0) {
                    hasBalances = true;
                    System.out.println(userA + " owes " + entry.getKey() + ": Rs." + entry.getValue());
                }
            }
        }
        if (!hasBalances) System.out.println("No balances due.");
    }
}
```

6 Case Study 4: Thread-Safe LRU Cache ($O(1)$ Get/Put)

The Least Recently Used (LRU) Cache requires $O(1)$ average time complexity for both get and put operations. This is achieved by combining a HashMap (for constant-time lookup) with a custom Doubly Linked List (for constant-time node promotion and eviction).

```
import java.util.*;
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class LRUCache<K, V> {
    private static class Node<K, V> {
        K key;
        V value;
        Node<K, V> prev, next;
        Node(K k, V v) { this.key = k; this.value = v; }
    }

    private final int capacity;
    private final Map<K, Node<K, V>> map = new HashMap<>();
    private final Node<K, V> head = new Node<>(null, null); // Dummy head
    private final Node<K, V> tail = new Node<>(null, null); // Dummy tail
    private final ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock();

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    private void removeNode(Node<K, V> node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    private void addFirst(Node<K, V> node) {
        node.next = head.next;
        node.prev = head;
        head.next.prev = node;
        head.next = node;
    }

    private void moveToFront(Node<K, V> node) {
```

```
        removeNode(node);
        addFirst(node);
    }

    public V get(K key) {
        rwLock.writeLock().lock(); // Write lock needed because get updates access order
        try {
            Node<K, V> node = map.get(key);
            if (node == null) return null;
            moveToFront(node);
            return node.value;
        } finally {
            rwLock.writeLock().unlock();
        }
    }

    public void put(K key, V value) {
        rwLock.writeLock().lock();
        try {
            Node<K, V> existing = map.get(key);
            if (existing != null) {
                existing.value = value;
                moveToFront(existing);
            } else {
                if (map.size() >= capacity) {
                    // Evict least recently used node (before dummy tail)
                    Node<K, V> lru = tail.prev;
                    removeNode(lru);
                    map.remove(lru.key);
                }
                Node<K, V> newNode = new Node<>(key, value);
                map.put(key, newNode);
                addFirst(newNode);
            }
        } finally {
            rwLock.writeLock().unlock();
        }
    }
}
```

7 Google Signature Low-Level Systems & Utilities

While consumer companies (Flipkart, Uber, Swiggy) frequently test business entity systems (Parking Lot, Food Delivery), Google's LLD interviews uniquely emphasize Systems and Data Structure Utilities. These three designs are Google's signature favorites.

7.1 1. Design an In-Memory Unix File System (Composite Pattern)

Problem: Implement an in-memory file system supporting ls(path), mkdir(path), addContentToFile(path, content), and readContentFromFile(path). Pattern: Composite Pattern with an N-ary tree / trie-like path hierarchy.

```
import java.util.*;

public class FileSystem {
    abstract static class Node {
        String name;
        abstract boolean isFile();
    }

    static class FileNode extends Node {
        StringBuilder content = new StringBuilder();
        boolean isFile() { return true; }
```

```
    }

    static class DirNode extends Node {
        Map<String, Node> children = new TreeMap<>(); // Sorted for ls()
        boolean isFile() { return false; }
    }

    private final DirNode root = new DirNode();

    private Node traverse(String path, boolean createDirs) {
        String[] parts = path.split("/");
        Node curr = root;
        for (String part : parts) {
            if (part.isEmpty()) continue;
            DirNode dir = (DirNode) curr;
            if (!dir.children.containsKey(part)) {
                if (!createDirs) return null;
                dir.children.put(part, new DirNode());
            }
            curr = dir.children.get(part);
        }
        return curr;
    }

    public List<String> ls(String path) {
        Node node = traverse(path, false);
        if (node == null) return Collections.emptyList();
        if (node.isFile()) return Collections.singletonList(node.name);
        return new ArrayList<>(((DirNode) node).children.keySet());
    }

    public void mkdir(String path) {
        traverse(path, true);
    }

    public void addContentToFile(String path, String content) {
        int lastSlash = path.lastIndexOf('/');
        String dirPath = path.substring(0, Math.max(1, lastSlash));
        String fileName = path.substring(lastSlash + 1);
        DirNode dir = (DirNode) traverse(dirPath, true);
        Node fileNode = dir.children.computeIfAbsent(fileName, k -> {
            FileNode fn = new FileNode();
            fn.name = fileName;
            return fn;
        });
        ((FileNode) fileNode).content.append(content);
    }

    public String readContentFromFile(String path) {
        Node node = traverse(path, false);
        if (node == null || !node.isFile()) throw new IllegalArgumentException("File not found!");
        return ((FileNode) node).content.toString();
    }
}
```

7.2 2. Transactional In-Memory Key-Value Store with Nested Transactions

Problem: Implement an in-memory key-value store with support for nested transactions: get(key), set(key, val), delete(key), begin(), commit(), and rollback(). Solution: Maintain a global committed state map and a Transaction Stack of active transaction frames (deltas).

```
import java.util.*;
```

```
public class TransactionalKVStore {
    private final Map<String, String> globalStore = new HashMap<>();
    private final Deque<Map<String, String>> txnStack = new ArrayDeque<>();

    public void begin() {
        txnStack.push(new HashMap<>()); // Push new transaction frame
    }

    public void set(String key, String value) {
        if (txnStack.isEmpty()) {
            globalStore.put(key, value);
        } else {
            txnStack.peek().put(key, value); // Record in active transaction frame
        }
    }

    public String get(String key) {
        // Inspect transaction frames from newest to oldest
        for (Map<String, String> frame : txnStack) {
            if (frame.containsKey(key)) {
                String val = frame.get(key);
                return (val == null) ? null : val; // null indicates deleted tombstone
            }
        }
        return globalStore.get(key);
    }

    public void delete(String key) {
        if (txnStack.isEmpty()) {
            globalStore.remove(key);
        } else {
            txnStack.peek().put(key, null); // Tombstone marker
        }
    }

    public boolean commit() {
        if (txnStack.isEmpty()) return false;
        Map<String, String> completed = txnStack.pop();
        if (txnStack.isEmpty()) {
            // Apply directly to global store
            for (Map.Entry<String, String> entry : completed.entrySet()) {
                if (entry.getValue() == null) globalStore.remove(entry.getKey());
                else globalStore.put(entry.getKey(), entry.getValue());
            }
        } else {
            // Merge into parent transaction frame
            txnStack.peek().putAll(completed);
        }
        return true;
    }

    public boolean rollback() {
        if (txnStack.isEmpty()) return false;
        txnStack.pop(); // Discard active transaction frame in O(1)
        return true;
    }
}
```

7.3 3. Thread-Safe Rate Limiter (Token Bucket Algorithm)

Problem: Implement a thread-safe, high-concurrency rate limiter allowing C burst capacity and refilling R tokens per second.

```
import java.util.concurrent.locks.ReentrantLock;

public class TokenBucketRateLimiter {
    private final long capacity;
    private final double refillRatePerMs;
    private double currentTokens;
    private long lastRefillTimestamp;
    private final ReentrantLock lock = new ReentrantLock();

    public TokenBucketRateLimiter(long capacity, double tokensPerSecond) {
        this.capacity = capacity;
        this.refillRatePerMs = tokensPerSecond / 1000.0;
        this.currentTokens = capacity;
        this.lastRefillTimestamp = System.currentTimeMillis();
    }

    public boolean allowRequest(int tokens) {
        lock.lock();
        try {
            refill();
            if (currentTokens >= tokens) {
                currentTokens -= tokens;
                return true;
            }
            return false;
        } finally {
            lock.unlock();
        }
    }

    private void refill() {
        long now = System.currentTimeMillis();
        long elapsedMs = now - lastRefillTimestamp;
        if (elapsedMs > 0) {
            currentTokens = Math.min(capacity, currentTokens + elapsedMs * refillRatePerMs);
            lastRefillTimestamp = now;
        }
    }
}
```

8 Daily 2-Hour Action Plan for Days 8 to 15

- Day 8 (2 hrs): Master ReentrantLock and tryLock(). Write a concurrent bank transfer program avoiding deadlocks.
- Day 9 (2 hrs): Code the Producer-Consumer pipeline using BlockingQueue and ExecutorService.
- Day 10 (2 hrs): Implement the Parking Lot System in your IDE. Test vehicle entry, spot allocation, and fee calculation.
- Day 11 (2 hrs): Extend the Parking Lot to multi-floors and implement an electric vehicle charging spot strategy.
- Day 12 (2 hrs): Implement BookMyShow concurrent seat reservation. Write a multi-threaded test simulating 10 users booking 1 seat.
- Day 13 (2 hrs): Implement Splitwise expense manager. Test equal, exact, and percentage splits.
- Day 14 (2 hrs): Code the custom LRU Cache using HashMap and Doubly Linked List from scratch without using LinkedHashMap.
- Day 15 (2 hrs): Implement Google's signature systems: the In-Memory File System and the Transactional Key-Value Store.